ECHO5 DIGITAL

BIG ED'S RETAIL OS

DEVELOPER KICKOFF

ARCHITECTURE + BUILD PLAN

Custom e-commerce | In-store POS | Inventory | Purchasing | Fulfillment



BUILD ONE CONNECTED SYSTEM.

Ship in verified milestones. Protect inventory and payment integrity.

ONE SHOWROOM + WAREHOUSE **POS REQUIRED**

Internal technical plan • Not a pricing proposal

October 2026 | Version 1.0

01 Scope and review decisions

Corrections made after reviewing the earlier preliminary development plan.

Confirmed delivery scope

- Public furniture storefront, branded around the approved Big Ed bear and slogan “Big Comfort. Bigger Savings.”
- One store operation, with showroom and warehouse modeled as separate stock locations.
- Desktop-first staff application: products, inventory, purchasing, orders, fulfillment, reports and role controls.
- In-store browser POS supporting configured card terminal integration, cash, deposits, receipts and returns.
- One shared backend, transactional inventory model and central order history.

Important corrections to previous draft

PAYMENT / POS	Select the provider and supported reader model early; validate terminal API, partial/deposit payments, refunds, failure recovery, webhooks and settlement reports.
INVENTORY	Do not use simple on-hand minus reserved alone. Account for damaged/quarantined stock, reservation expiry, fulfillment and movement auditability.
ORDERS	Separate order state, payment state and fulfillment state. Never mark a sale complete solely because a checkout page displayed success.
ACCOUNTING / TAX	Use a supported tax calculation approach; tax applicability and delivery policies need business sign-off. Accounting export is optional MVP unless required.
BRANDING	Client storefront photo is reference; obtain a clean logo, mascot variations and proof of commercial usage rights before final artwork.

Scope guardrail: no native Windows executable, offline POS, advanced AI, automated vendor APIs, multi-store UI, complex delivery routing or AR in MVP.

02 Platform architecture

Modular monolith + shared PostgreSQL, deployed as separate experiences.

CUSTOMER WEB

Next.js storefront

Shop, cart, checkout

DESKTOP ADMIN

Next.js operations

Catalog, PO, stock, reports

IN-STORE POS

Next.js cashier app

Sales, deposits, refunds

↓ SECURED HTTP APIs ↓

UNIFIED NESTJS API / MODULAR MONOLITH

Auth + RBAC | Catalog | Pricing | Inventory | Orders

Payments | POS Sessions | Purchasing | Receiving

Fulfillment | Audit + Reporting | Notifications

PostgreSQL + Prisma

Transactional business data

Object Storage + CDN

Product / brand images

External Providers

Payments, taxes, email / SMS

Key architecture decisions

- pnpm workspace monorepo: apps/storefront, apps/admin, apps/pos, apps/api; packages/database, ui, types, validation, config.
- Backend authorization must be enforced on every staff endpoint. No business-critical stock calculations in the browser.
- Managed PostgreSQL and storage with backups, tested restores, staged environments and secure secret management.
- Use outbox / idempotent workers for webhook-driven notifications and payment updates. Prevent duplicate events and duplicate inventory movements.
- Keep product search simple for MVP (indexed database query); upgrade to a search engine only if catalog size or requirements justify it.

03 Source-of-truth workflows

Explicit state machines prevent silent business-data corruption.

Order lifecycle - separate dimensions

Dimension	Example states
Order	Draft / Confirmed / Canceled / Closed
Payment	Unpaid / Partially paid / Paid / Refunded
Fulfillment	Unallocated / Reserved / Ready / Picked up or Delivered

Business-critical flows

ONLINE CHECKOUT

Cart → inventory reservation with expiration → payment confirmation via server/provider event → confirmed order → pick/ship or pickup.

POS CARD SALE

Register open → scan items → create order/reservation → terminal payment → verify provider outcome → receipt and confirmed sale.

POS CASH / DEPOSIT

Authorized cashier records cash or deposit, prints receipt, tracks remaining balance and inventory commitment; closing reconciles tender totals.

RECEIVING STOCK

Approved PO → receiving inspection → accepted quantity recorded exactly once → movement ledger + balance update → discrepancy log.

RETURN / REFUND

Original order → return authorization → inspect item → restock/quarantine decision → provider refund (if applicable) → audit trail.

04 Data model and technical guardrails

Schema design should be reviewed before implementation starts.

Initial domain model

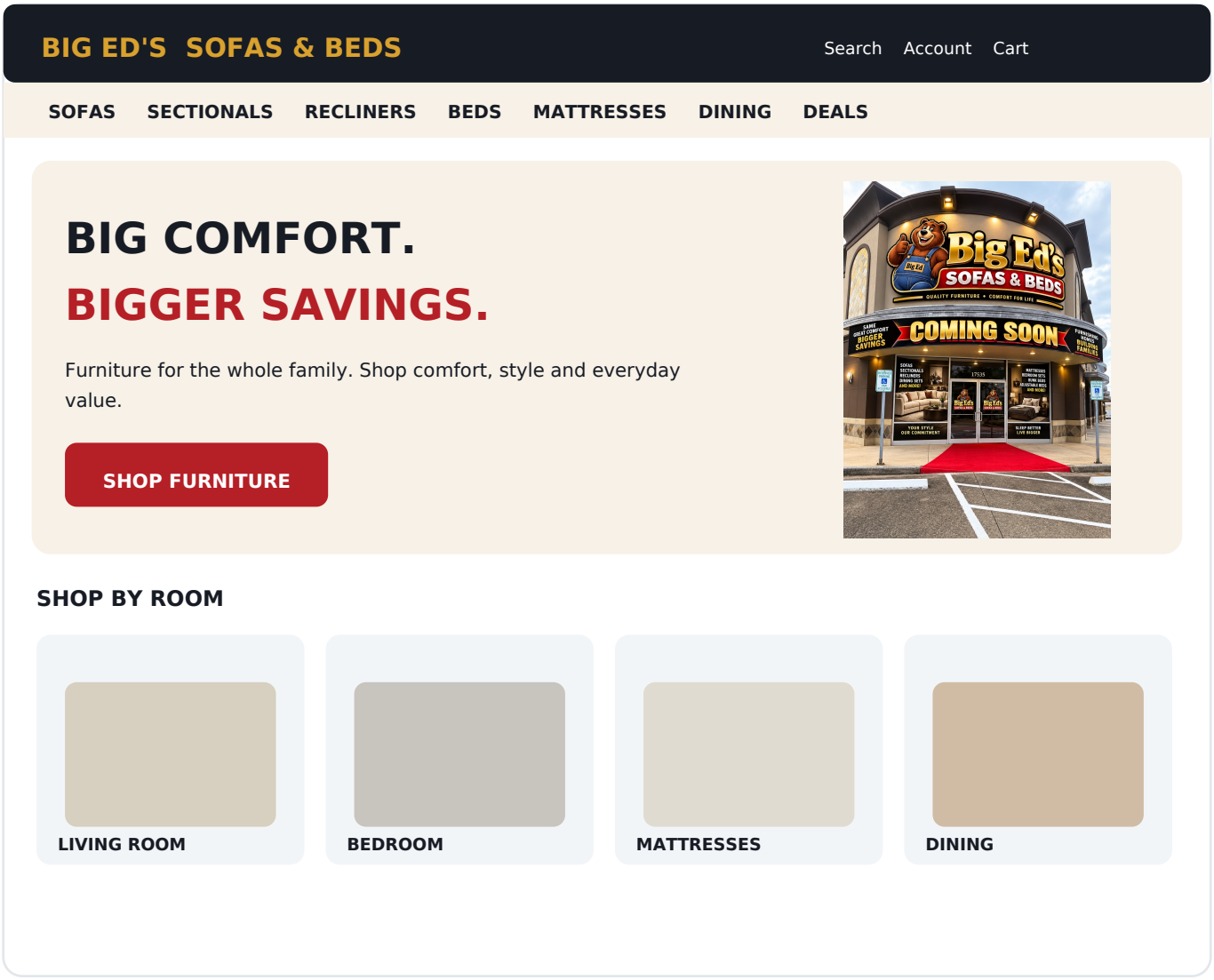
Domain	Required entities
Auth	User, Role, Permission, UserRole, Session, AuditLog
Products	Product, Variant, Category, ProductImage, PriceHistory
Stock	Location, StockBalance, StockMovement, Reservation, Transfer, Count
Sales	Customer, Address, Order, OrderLine, Payment, Refund, ProviderEvent
POS	Register, RegisterSession, CashMovement, Receipt
Purchasing	Supplier, PurchaseOrder, POLine, GoodsReceipt, ReceiptLine
Delivery	Fulfillment, Delivery, Pickup, StatusHistory

Non-negotiable engineering rules

- Use UUID/ULID IDs, uniqueness constraints on SKU and payment/provider event IDs, foreign keys and purposeful indexes.
- Record item prices, taxes and discounts as historical order snapshots. Store USD values as cents or safe decimal, not floating-point.
- Available-to-sell = physical on-hand - active reservations - quarantined/unsellable allocation. Define actual stock deduction point.
- Use database transactions and concurrency control for final-unit sales, transfers, receipts and reservation conversions.
- Never assume terminal “pending” means “paid”; payment confirmation and provider event idempotency are required.
- Immutable movement history, reversible adjustments via compensating events, immutable audit trail of privileged actions.

05 Front-end concept: customer website

Illustrative interface direction - not an approved final design.

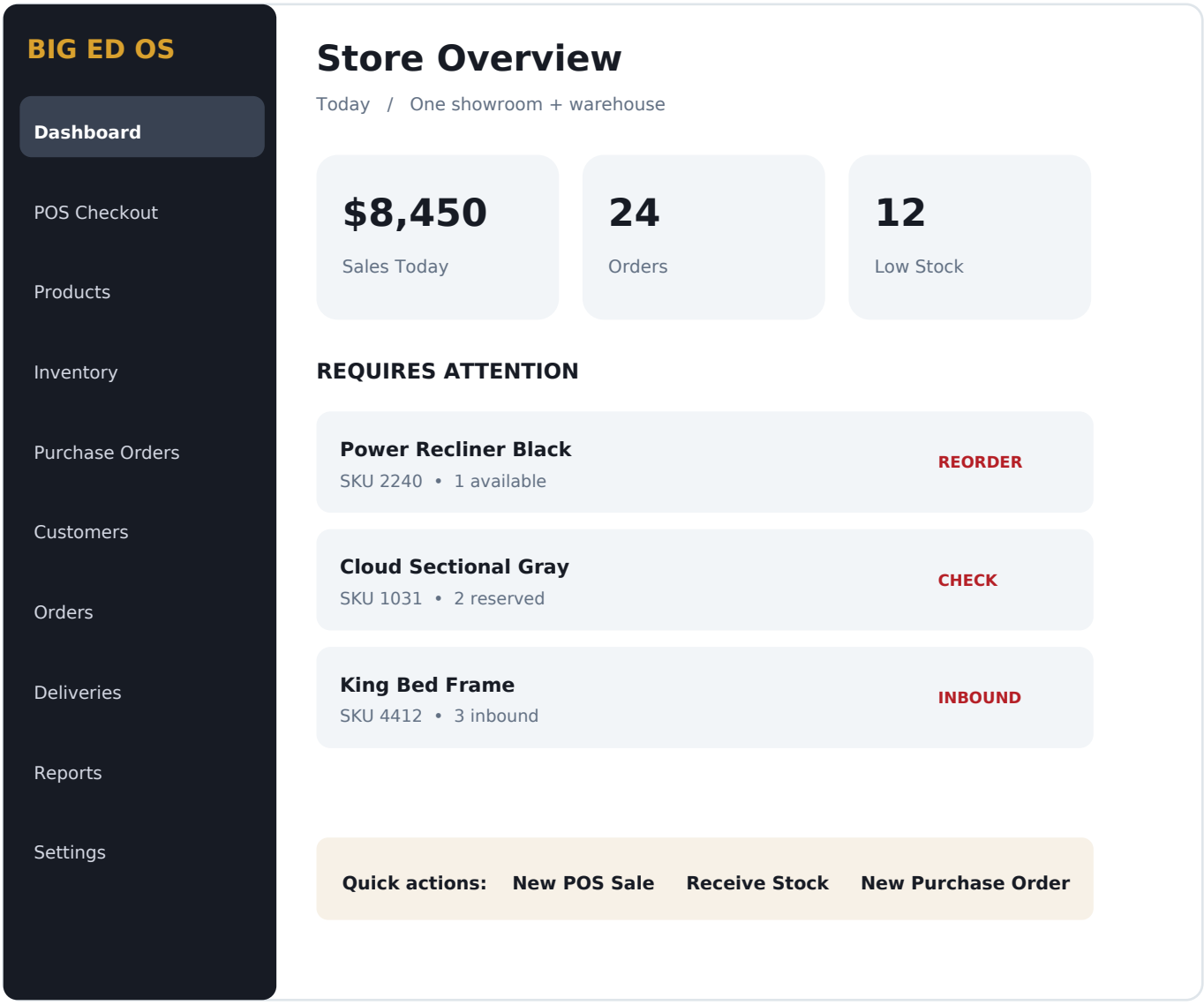


Design details:

- Mascot appears in hero/promotions/help - not on every product thumbnail. Use the client-approved illustration files, not the storefront photo as a logo.
- Search, categories, transparent availability, accurate dimensions, pickup and local delivery should be prominent on every product detail page.
- Mobile-first: large tap targets, sticky Add to Cart, compressed product photos, accessible contrast and clear fees before checkout.

06 Front-end concept: retail dashboard

Desktop-first operations UI - fast workflows and role-aware navigation.

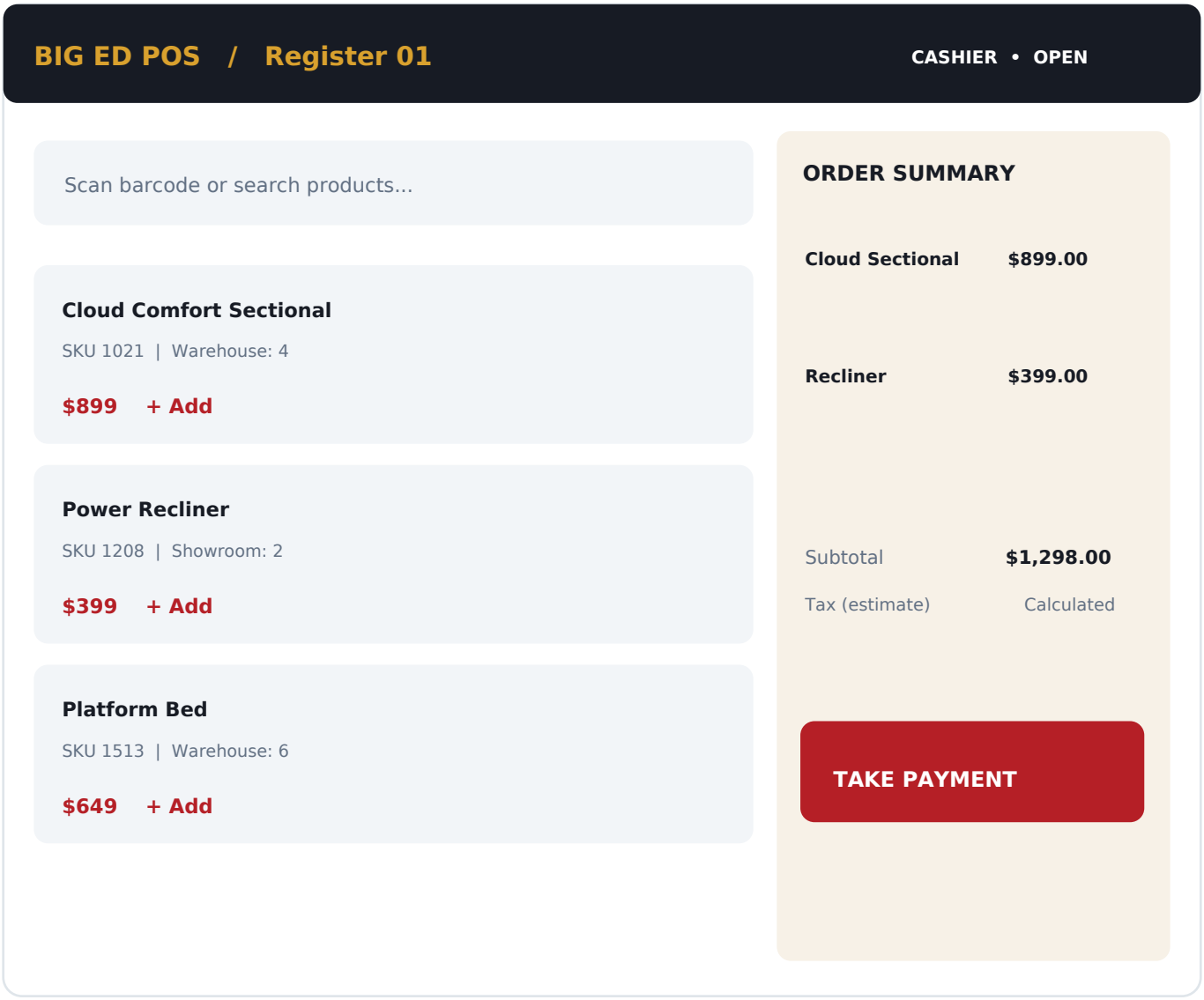


Illustrative sample data. Role permissions drive which financial and purchasing information is visible to each employee.

- Sidebar modules: Dashboard, POS, Catalog, Inventory, Purchasing, Orders, Fulfillment, Customers, Reports and Settings.
- Design for 1366px-and-larger office screens, with functional tablet breakpoints; high-density tables with keyboard navigation and barcode scanner input.
- Show stock by location, reserved units, reorder status and last activity; require reason codes for adjustments, discounts and refunds.

07 Front-end concept: cashier POS

Checkout workflow should minimize clicks without bypassing payment verification.



- Search / scan; stock by location; cart quantity control; customer attach; discounts only if permitted; pickup/delivery choice.
- Select payment tender: cash, supported card-present terminal, or deposit. Handle partial/failed payments; generate receipt only after confirmed success.
- Cashier closeout reports must reconcile cash collected, card provider outcomes, refunds and recorded discrepancies.

08 Purchasing, receiving and operational screens

Supplier purchase orders and receiving are part of core inventory integrity.

PURCHASE ORDERS + CREATE NEW PO

PO NUMBER	SUPPLIER	EXPECTED	STATUS
PO-1048	Comfort Distribution	Oct 20	Partially received
PO-1049	Apex Furniture Co.	Oct 23	Awaiting approval
PO-1050	RestWell Wholesale	Oct 25	Sent to vendor

MVP purchasing actions

- Supplier profiles, cost, expected lead time and purchase order draft/approval status.
- PO line items include exact SKU, expected quantity, supplier unit cost, tax/freight if applicable and receiving location.
- Receiving workflow supports partial deliveries, damaged shipments, difference notes and repeated receipts without over-receiving.
- Low-stock report suggests purchase quantities; manager must approve before emailing vendor.
- Exportable PDF POs and purchase history; supplier APIs and automatic buying are future enhancements.

Order / delivery console

- Filter by new, paid, reserved, ready for pickup, scheduled delivery, completed, canceled.
- Show customer contact, item dimensions and assembly notes; capture delivery area/fees and agreed date.
- Reserve items for orders with deposits according to manager-approved policy and reservation expiry rules.

09 Milestones and acceptance criteria

Target: approximately 18–22 weeks. Adjust after vendor/payment discovery.

Milestone	Target	Acceptance / evidence
0: Discovery	Week 1–2	Hardware/provider feasibility, master requirements, product fields, brand assets, architecture approval.
1: Foundation	Week 2–4	Repo, CI, auth/RBAC, schema, migrations, seeded catalog, staged UI shells.
2: Inventory	Week 5–8	Catalog CRUD, stock ledger, reservations, receiving, adjustment, transfer and concurrency tests.
3: Purchasing	Week 8–11	Supplier, PO, approvals, partial receipts and stock impact verified.
4: POS	Week 10–15	Cashier checkout, cash/card, deposits, terminal payment flows, refunds, register reconciliation.
5: E-commerce	Week 11–16	Branded product pages, filters, cart, checkout, inventory integration, order tracking.
6: Pilot + launch	Week 17–22	Import, security/QA, provider reconciliation, full pilot, staff training, rollback plan.

Quality gates per milestone

- Working staging demo, reviewed pull requests, passing lint/typecheck/unit/integration tests and documented migration status.
- Written sign-off on acceptance scenarios; screenshots and videos are evidence but never replace transaction-level tests.
- Cross-channel test: last unit ordered concurrently through POS and web is never sold twice.
- Financial test: payment fail/retry/refund/webhook duplicates do not create phantom sales or double stock movements.
- Operational test: partial goods receipt updates only accepted quantities; cashier closeout reconciles against payment records.

10 Immediate Sprint 1 - assign today

First ten working days: foundation and validated assumptions, not full implementation.

DAYS 1-2 — **Project setup**

- Create GitHub repo, pnpm workspace and code owners; CI checks.
- Create docs/requirements, docs/architecture, docs/workflows, CLAUDE.md.
- Record confirmed owner, roles, payment provider and POS hardware as decisions or blockers.

DAYS 3-5 — **Schema + security**

- Design Prisma schema and migrations with constraints and indexes.
- Initialize local Postgres, test/dev seed fixtures and migration validation.
- Implement secure auth, protected staff routes and initial RBAC.

DAYS 6-8 — **Catalog + UI skeleton**

- Build APIs for categories and product variants (no stock writes from storefront).
- Create product list/editor in admin and shared brand UI components.
- Implement homepage shell with approved brand assets or safe placeholders.

DAYS 9-10 — **Demo + sign-off**

- Demonstrate seeded products in admin and website preview.
- Run builds, tests, login/permission checks and migration clean install.
- Deliver tested staging URLs, changelog, risks and Sprint 2 ticket breakdown.

11 Claude Code workflow and release rules

Human engineering ownership remains mandatory.

Recommended repository organization

```
apps/storefront  apps/admin  apps/pos  apps/api

packages/database  packages/ui  packages/types  packages/validation

docs/architecture  docs/database  docs/workflows  docs/requirements

tests/e2e  assets/brand  CLAUDE.md  pnpm-workspace.yaml
```

Rules developers should put into CLAUDE.md

- Read existing specs and propose a small implementation plan before changing code.
- One isolated task/branch or PR at a time; ask before introducing major dependencies or changing schema design.
- Generate tests for business-critical actions, execute them, and report actual failures rather than asserting success.
- Never insert real credentials, client payment data or production customer records into AI prompts/repository.
- No production migration, payment test with live funds or deployment without designated human approval.
- Require PR review, migration review, smoke-test and staging approval before promoting a build.

Suggested first Claude Code task

```
Inspect the repository. First produce docs/architecture/overview.md, docs/database/schema.md, docs/requirements/open-questions.md and an actionable Sprint 1 task list. Then scaffold the monorepo, secure staff authentication and basic product management. Stop before integrating payment processing or implementing the inventory ledger; those require reviewed specifications and test scenarios. Run lint, typecheck, tests and builds and report exact commands/results.
```

12 Required client inputs + launch gate

The project can start before every decision is finalized, but not all features can.

Ask client / manager in Week 1

- Current POS system or preferred payment processor, merchant account and in-store card terminal models.
- Starter product catalog (SKU, variants, photos, brands, dimensions, costs, prices, current quantities).
- Warehouse/showroom physical layout and receiving process; stock count owner.
- Deposit policy, cancellations, refunds, exchanges, delivery/pickup rules and staff discount authorization.
- Sales-tax jurisdiction and desired finance/accounting integration, if any.
- Brand source files and written rights to logo/mascot artwork.
- Access control and ownership for domain, hosting, payment account, email/SMS services.

Mandatory production launch checklist

- Approved production test cases for checkout, refunds, failed payments, duplicate webhook events and concurrent stock reservation.
- Inventory migration is verified through physical count and reconciliation before enabling checkout.
- Payment terminals and payment webhooks tested on actual supported hardware and provider sandbox/live certification process.
- Automated backup and tested restore; monitoring, alerting, roles, permissions and incident escalation configured.
- Staff can complete an end-to-end POS sale, PO receipt, local delivery/pickup and register closeout without developer assistance.
- Documented go/no-go decision, rollback window, launch-day owner and support coverage.

FIRST APPROVAL REQUIRED

Approve Sprint 1 technical scope and assign responsible owners. Do not begin irreversible payment/provider work or production data migration until the corresponding prerequisites are confirmed.

13 Team handoff: ownership and definition of done

Use this page to organize the first development kickoff meeting.

Role	Primary responsibility
Tech lead	Architecture, schema reviews, provider feasibility, release decisions
Backend engineer	API modules, inventory ledger, payment and PO rules
Frontend engineer	Storefront, admin and POS interaction patterns
Designer	Mascot assets, responsive layout, product UX and accessibility
QA / delivery lead	Acceptance cases, CI evidence, launch rehearsal and training

Definition of done for every completed ticket

- Requirements and acceptance scenario linked to the PR.
- Code reviewed; validation, RBAC and error handling implemented where relevant.
- Passing tests, typecheck, build and database migrations where applicable.
- Documentation and change log updated; staging demo available if UI-facing.
- No fabricated third-party capability or unverified production readiness claims.

START WITH THE FOUNDATION.

A great-looking storefront will not save an unreliable checkout or inventory system. First confirm payments and workflows, then build the shared catalog, permissions and stock ledger. Ship the UI in parallel using approved Big Ed brand assets. Expand features only after transaction-level acceptance tests pass.